

Petit Rayon de Soleil

Application web de messages
bienveillants, sécurisée et modérée

SOUTENANCE DE PROJET

Concevoir, développer, tester et déployer une application web complète.

Hélio de Breyne | MyDigitalSchool | Session CDA | 2026



Une présentation conçue pour 40 minutes

Le temps de démonstration est réservé au cœur de l'application.



Règle de conduite

- Ne pas lire les diapositives : elles soutiennent le discours.
- Privilégier les décisions et les preuves plutôt que la liste des technologies.
- Garder la démonstration pour montrer les parcours les plus importants.

Profil, formation et environnement professionnel

Un projet de formation enrichi par une expérience des usages numériques.

Mon parcours

- Formation : Concepteur développeur d'applications.
- École : MyDigitalSchool.
- Progression par projets, pratique et mise en situation.
- Objectif : construire des applications utiles, maintenables et défendables.

Expérience en entreprise : In Square

- Environnement tourné vers les usages numériques et les contenus.
- Sensibilisation à la lisibilité, à l'expérience utilisateur et à la qualité perçue.
- Réflexion sur la cohérence entre contenu, interface et public.

Le projet Petit Rayon de Soleil relie conception, front end, back end, sécurité, tests et déploiement dans un seul produit.

Comment proposer un espace positif sans publication non maîtrisée ?

La simplicité fonctionnelle ne supprime pas les règles de sécurité et de modération.

Créer une application web qui permet de consulter et de proposer des messages bienveillants, tout en empêchant la publication immédiate de contenus non validés.



Expérience simple

Un parcours rapide et accessible, sur desktop comme sur mobile.



Contribution encadrée

Un membre peut proposer, mais ne publie pas directement.



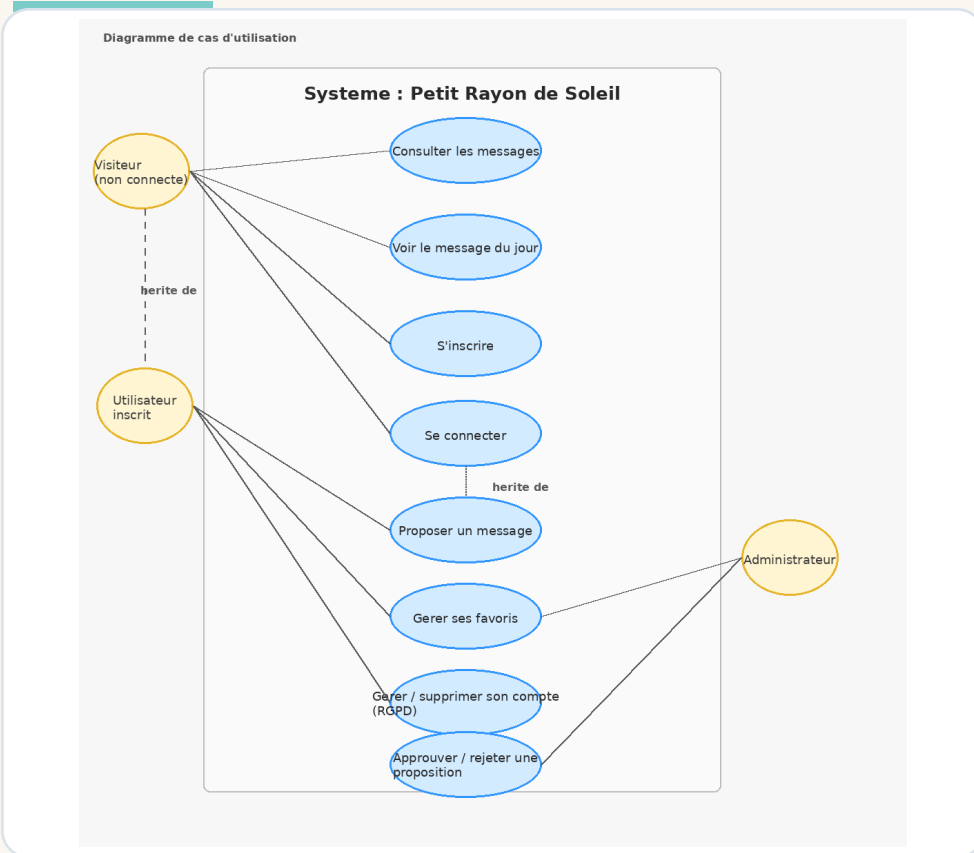
Modération traçable

Un administrateur contrôle la validation avant diffusion.

Le choix de la modération structure les rôles, les données, les règles métier et les tests.

Trois rôles pour une application lisible et contrôlée

Chaque rôle possède des droits cohérents avec le parcours prévu.



Visiteur

Consulte les messages validés et découvre le service.



Utilisateur inscrit

Crée un compte, propose un message, gère ses favoris et son profil.

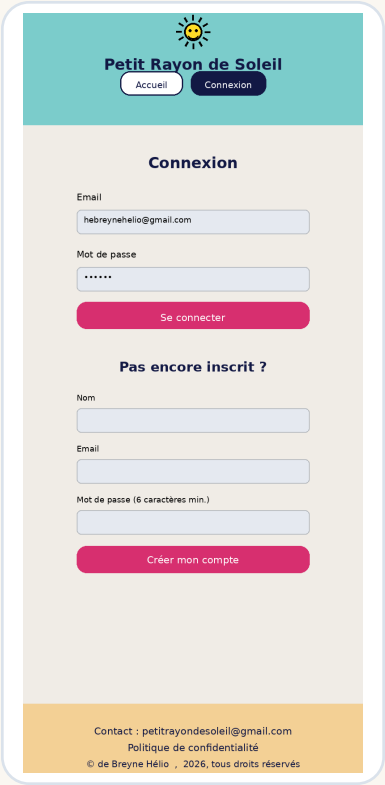


Administrateur

Valide ou rejette les propositions avant leur publication.

Une interface simple, douce et responsive

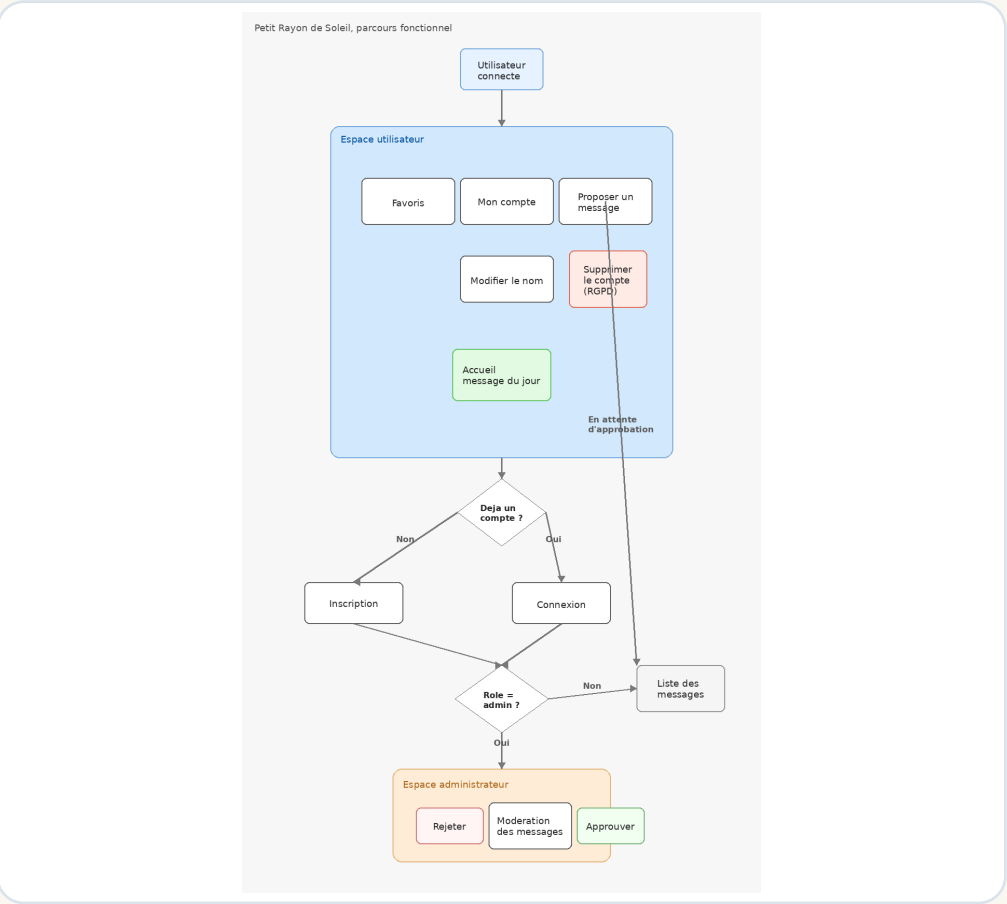
Les écrans rendent les actions principales immédiatement identifiables.



Navigation courte • hiérarchie visuelle claire • appels à l'action visibles • adaptation aux petits écrans

Du message proposé au message publié

Le workflow rend visible la règle centrale : la modération avant publication.



- 1

Consulter

Le visiteur lit les messages déjà validés.
- 2

S’authentifier

Le membre crée un compte puis ouvre sa session.
- 3

Proposer

La proposition est enregistrée avec un statut en attente.
- 4

Modérer

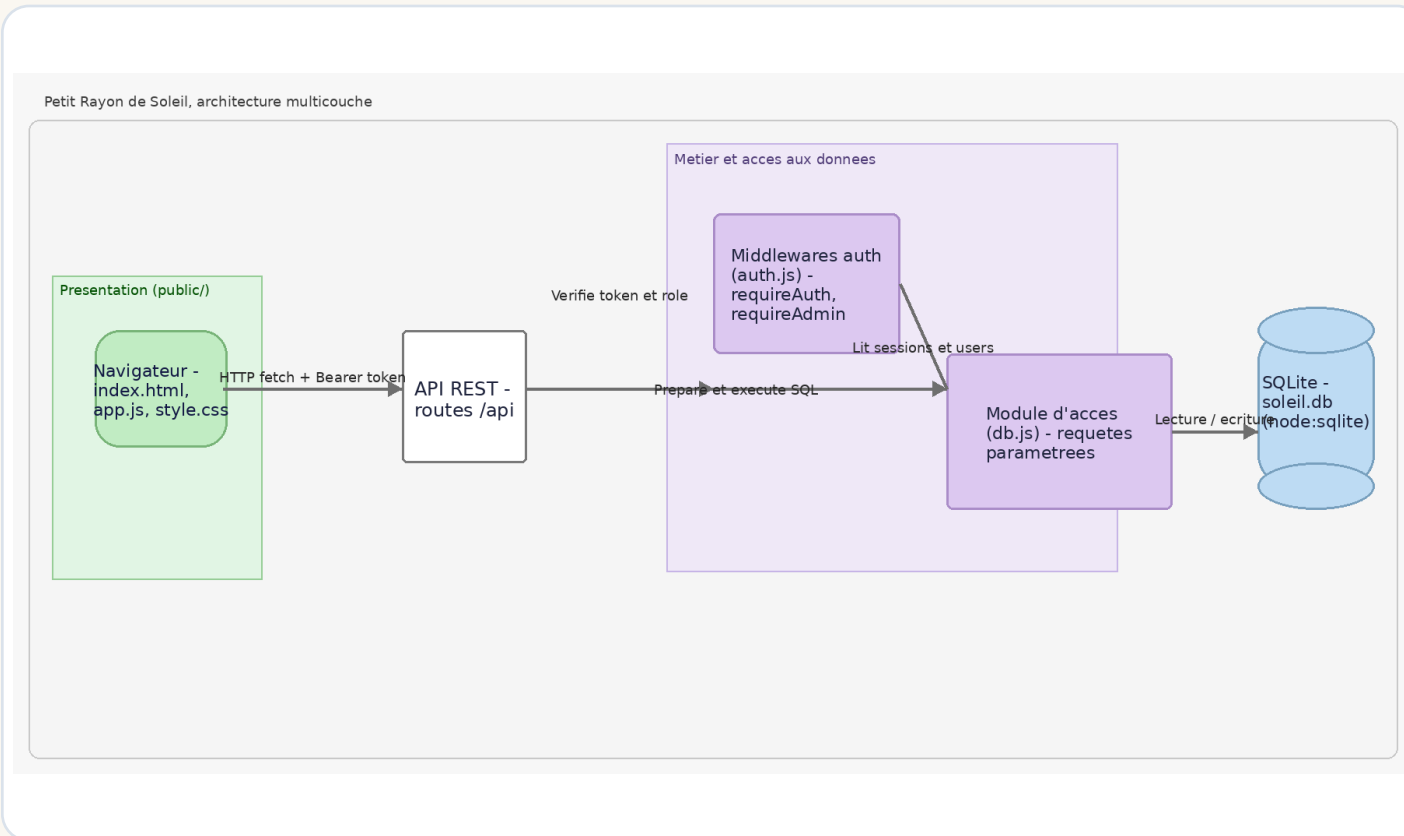
L’administrateur approuve ou rejette le contenu.
- 5

Publier

Seuls les messages approuvés deviennent visibles.

Une application organisée en couches

La séparation des responsabilités simplifie la maintenance, la sécurité et les tests.

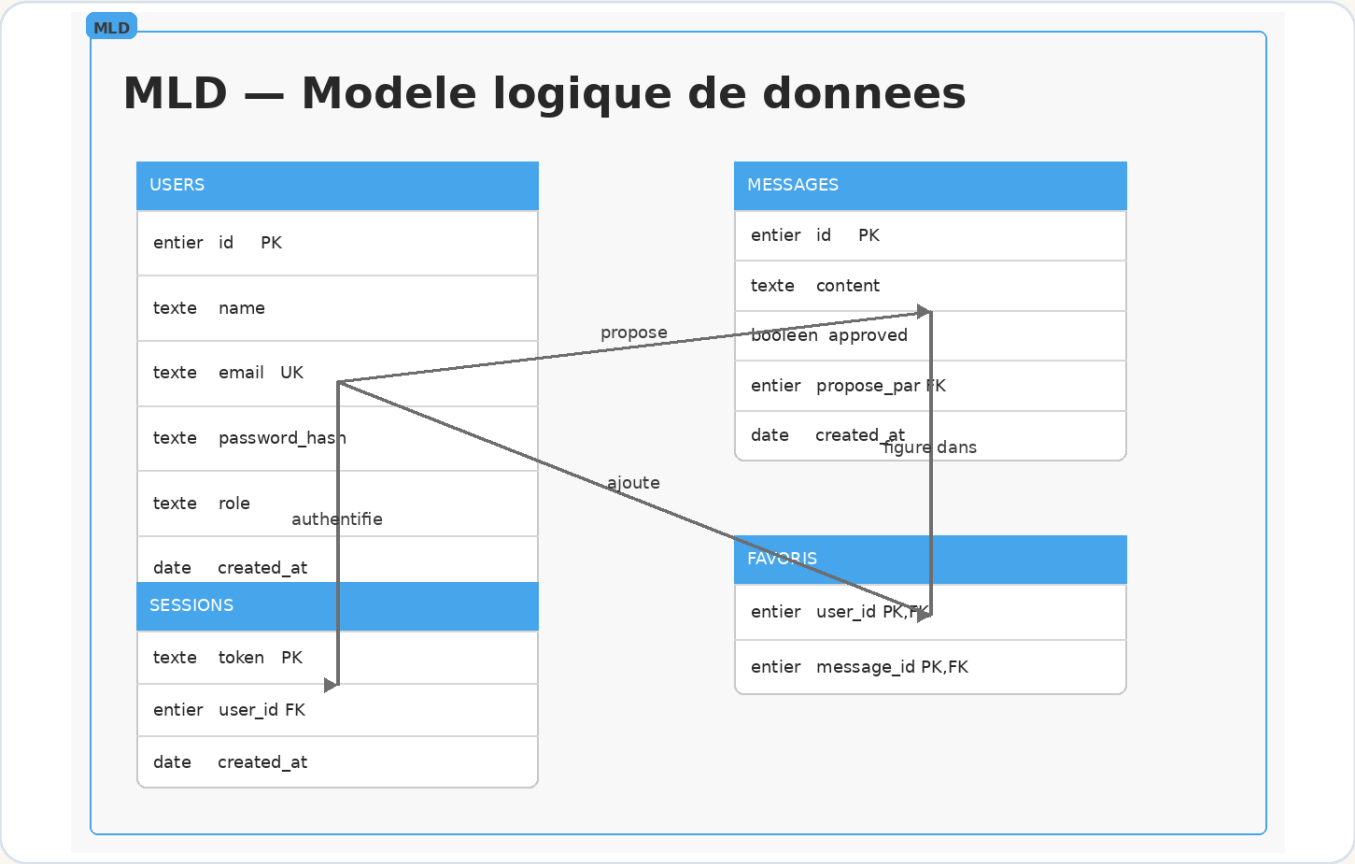


Chaîne applicative

- Front end : HTML, CSS et JavaScript.
- API : Node.js et Express, routes REST.
- Sécurité : middlewares d'authentification et de rôle.
- Données : SQLite en local, persistance compatible production.
- Déploiement : publication sur Vercel.

Un modèle relationnel réduit, mais structurant

Les entités traduisent les règles de gestion des comptes, messages, favoris et sessions.



-  **users** Identité, email unique, mot de passe haché, rôle.
-  **sessions** Maintien de la connexion et rattachement utilisateur.
-  **messages** Contenu, auteur, état de validation et date.
-  **favoris** Lien entre un utilisateur et un message publié.

Protéger les comptes, les routes et les données

La sécurité est traitée au niveau des accès, des entrées et du cycle de vie du compte.

02 Middleware requireAuth

auth - extrait

```
19  /** Middleware : exige un utilisateur connecté. */
20  function requireAuth(req, res, next) {
21    const user = userFromToken(tokenFromReq(req));
22    if (!user) return res.status(401).json({ error: 'Non authentifié' });
23    req.user = user;
24    next();
25  }
```

03 Contrôle administrateur

auth - extrait

min

```
19  /** Middleware : exige un utilisateur connecté ET administrateur. */
20  function requireAdmin(req, res, next) {
21    const user = userFromToken(tokenFromReq(req));
22    if (!user) return res.status(401).json({ error: 'Non authentifié' });
23    if (user.role !== 'admin') return res.status(403).json({ error: 'Accès réservé à l'administrateur' });
24    req.user = user;
25    next();
26  }
```

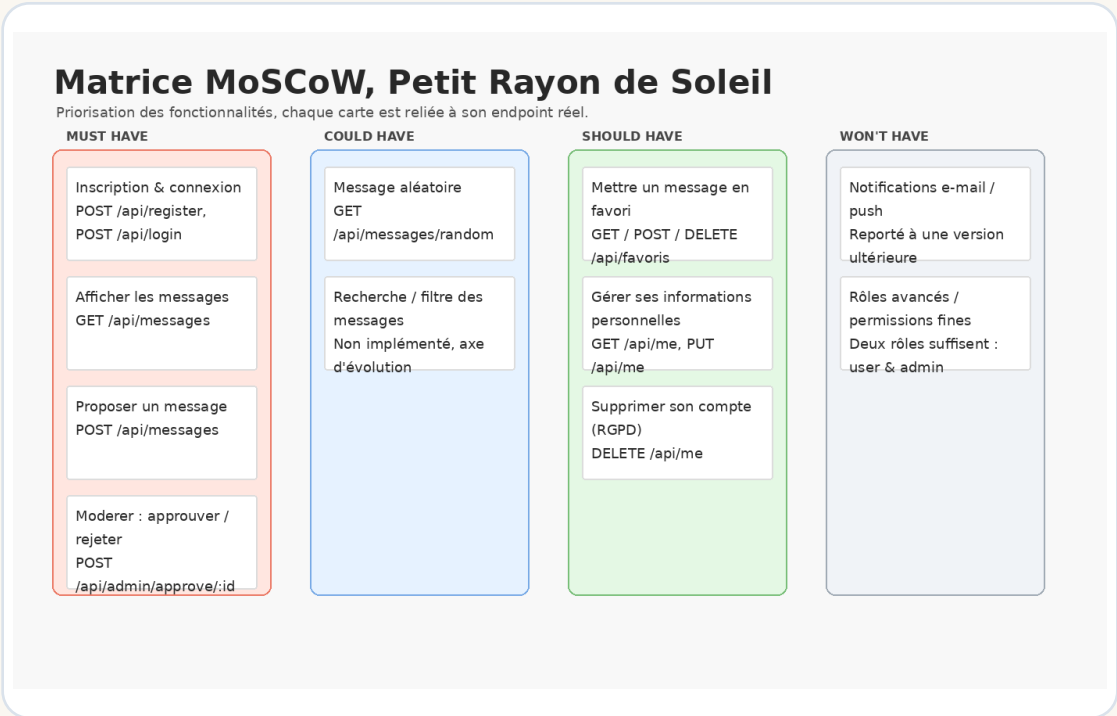
Mesures mises en œuvre

- Mots de passe hachés avec bcrypt.
- Session contrôlée avant accès aux routes protégées.
- Restriction des routes d'administration au rôle admin.
- Requêtes SQL paramétrées pour réduire le risque d'injection.
- Suppression du compte et des données associées dans une logique RGPD.

Principe clé : le serveur contrôle le droit réel de l'utilisateur, pas seulement l'interface.

Prioriser pour livrer un périmètre cohérent

Kanban et MoSCoW rendent visible l'avancement et les arbitrages.



Arbitrage assumé

- Sécuriser les fonctionnalités cœur, conserver un front end vanilla et reporter les évolutions non indispensables : notifications, recherche avancée et rôles complexes.

Des scénarios automatiques sur les parcours critiques

Le rapport d'exécution confirme la validation des fonctionnalités essentielles.

```
● ● ● petit-rayon-de-soleil , npm test

$ npm test
> petit-rayon-de-soleil@1.0.0 test
> node --test

Base initialisée avec un admin, un utilisateur et 5 messages.
✓ GET /api/messages/random renvoie un message (144.66ms)
✓ inscription puis connexion fonctionnent (293.67ms)
✓ connexion avec mauvais mot de passe est refusée (119.81ms)
✓ proposer un message sans être connecté est refusé (401) (4.42ms)
✓ un utilisateur normal ne peut pas accéder à l'admin (403) (120.08ms)
✓ parcours complet : proposition -> approbation admin (245.25ms)
✓ suppression de compte (RGPD) efface l'utilisateur (125.88ms)
i tests 7
i pass 7
i fail 0
i duration_ms 1745.69
```

7 / 7

tests réussis

0

échec signalé

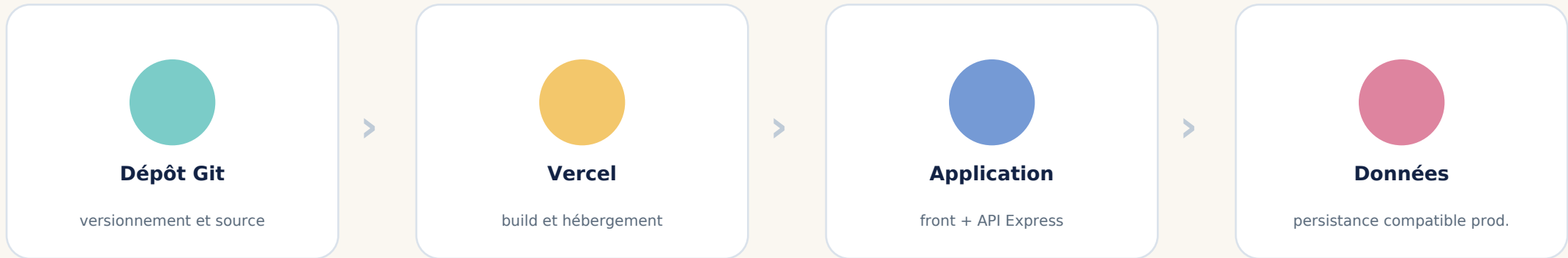
Scénarios couverts

- Inscription et connexion.
- Refus d'accès sans droit.
- Modération.
- Suppression de compte.

Les tests automatisés apportent une preuve répétable que les cas critiques sont vérifiés.

Une mise en ligne pensée pour un contexte serverless

Le projet est déployé sur Vercel, avec une configuration adaptée à la production.



Points de vigilance

- Séparer les paramètres locaux et les variables de production.
- Vérifier les routes, les autorisations et la persistance après publication.
- Prévoir linting, tests à chaque push et préproduction.

Montrer le produit sans perdre le fil

La démonstration valide les règles métier, elle ne remplace pas l'explication technique.

01

Accueil public

Afficher le message du jour et la consultation libre.

02

Connexion

Ouvrir une session avec un utilisateur de démonstration.

03

Proposition

Soumettre un message puis signaler son statut en attente.

04

Administration

Valider ou rejeter la proposition avec le rôle admin.

05

Vérification

Retourner sur l'accueil et vérifier la publication.

Prévoir un plan B : captures d'écran et rapport de tests si la démonstration rencontre un problème.

Un projet complet à l'échelle d'un produit de formation

Le produit relie besoins, conception, implémentation, validation et publication.

3

rôles métiers

Visiteur, utilisateur, administrateur

7 / 7

tests réussis

Parcours critiques automatisés

1

application déployée

Publication sur Vercel

5

espaces principaux

Accueil, connexion, favoris, profil,
proposition

Compétences CDA illustrées

- Développer une application sécurisée.
- Concevoir une application en couches.
- Préparer le déploiement : tests, configuration, publication et maintenabilité.

Un socle fonctionnel, des évolutions identifiées

Les limites montrent la capacité à prioriser les prochaines étapes.

Ce qui est acquis

- Parcours utilisateur complet.
- Modération avant publication.
- Architecture lisible, sécurité appliquée et tests automatisés.
- Déploiement sur Vercel.

Limites actuelles

- Pas de notifications e-mail ou push.
- Recherche avancée non implémentée.
- Rôles limités à user et admin.
- Industrialisation perfectible.

Prochaines évolutions

- Pipeline CI avec tests à chaque push.
- Préproduction et supervision.
- Amélioration de la modération.
- Recherche, catégories et personnalisation.

Choix assumé : livrer un cœur de produit robuste avant d'élargir les fonctionnalités.

CONCLUSION

Un projet conçu comme un produit complet.

Petit Rayon de Soleil m'a permis de mettre en œuvre un cycle complet de développement : analyser le besoin, concevoir les parcours et les données, développer une application sécurisée, la tester et préparer son déploiement.

Merci pour votre attention. Questions ?

Références : dossier de projet, captures d'écran, diagrammes et rapport de tests « Petit Rayon de Soleil », 2026.